

Análise Experimental de Desempenho e Exaustão de um Controlador SDN Baseado em Sockets TCP

Iago Roberto Esmério Almeida

¹Programa de Pós-Graduação em Ciência da Computação (PPGCC)
Universidade Federal do Piauí (UFPI) – Teresina, PI – Brasil

`iago.almeida@ufpi.edu.br`

Abstract. *We stress a minimalist single-threaded OpenFlow 1.0 controller written in Python over TCP sockets, with polling and a synchronous handshake, using Mininet (`tree, depth=2, fanout=4`) and cbench. Concurrent connections raise handshake latency to 4.3 s with 16 switches; under PACKET_IN flooding it saturates at about 30 thousand messages/s with CPU above 97% and up to 100% loss; cbench peaks at 108 thousand responses/s and fails with eight switches. Of three concurrent refactorings, all remove the connection bottleneck, but only selectors-based I/O multiplexing also improves latency-mode throughput, which explains why frameworks such as Ryu are asynchronous and event-driven. Video: [\[public link\]](#).*

Resumo. *Este trabalho estressa um controlador OpenFlow 1.0 minimalista em Python, com uma thread, polling e handshake síncrono sobre sockets TCP, usando o Mininet (`tree, depth=2, fanout=4`) e o cbench. Conexões simultâneas elevam a latência de handshake a 4,3 s com 16 switches; sob inundação de PACKET_IN, o controlador satura em cerca de 30 mil mensagens/s, com CPU acima de 97% e perda de até 100%; no cbench, atinge 108 mil respostas/s e falha com oito switches. Das três refatorações concorrentes, todas eliminam o gargalo de conexão, mas só a multiplexação de E/S com selectors também melhora a vazão no modo latência, o que explica por que frameworks como o Ryu são assíncronos e orientados a eventos. Vídeo: [\[link público\]](#).*

1. Introdução

As Redes Definidas por Software (*Software-Defined Networking*, SDN) separam o plano de dados, responsável pelo encaminhamento de pacotes nos switches, do plano de controle, que passa a ser exercido por um software logicamente centralizado, o controlador [Kreutz et al. 2015]. A comunicação entre os dois planos segue um protocolo aberto, sendo o OpenFlow o mais difundido [McKeown et al. 2008, Open Networking Foundation 2009]: o switch abre uma conexão TCP com o controlador, ambos negociam a versão do protocolo (HELLO) e as capacidades do switch (FEATURES_REQUEST/FEATURES_REPLY) e, a partir daí, todo pacote sem regra correspondente na tabela de fluxos (*table-miss*) é encaminhado ao controlador em uma mensagem PACKET_IN, que decide o que fazer com ele, por exemplo, com um PACKET_OUT ou com a instalação de uma regra (FLOW_MOD).

Essa centralização tem um custo: o controlador entra no caminho crítico de cada novo fluxo, e sua capacidade de processar mensagens passa a limitar o desempenho de toda a rede. Não por acaso, a vazão de PACKET_IN por segundo e a

latência de resposta tornaram-se as métricas usuais de comparação entre controladores [Tootoonchian et al. 2012, Erickson 2013, Zhu et al. 2019], e os controladores de produção, como Beacon [Erickson 2013], OpenDaylight [Medved et al. 2014], ONOS [Berde et al. 2014] e Ryu [Ryu SDN Framework Community 2026], investem em E/S assíncrona, processamento em lote e paralelismo para sustentar milhares de switches e milhões de eventos por segundo.

Este trabalho estuda o extremo oposto desse espectro: o `Controlador-v2.py`, um controlador didático de cerca de 230 linhas em Python puro que implementa o OpenFlow 1.0 diretamente sobre sockets TCP. Sua arquitetura é propositalmente minimalista: uma única thread; um laço que varre sockets não bloqueantes por *polling*, sem *select*, *selectors* ou *asyncio*; um handshake síncrono dentro desse laço, de modo que, enquanto um switch negocia, nenhum outro é atendido; uma função `recv_exact()` que gira em espera ocupada até completar cada mensagem; e um comportamento de *hub*, que responde a todo `PACKET_IN` com um `PACKET_OUT` de inundação sem instalar regras. Assim, todo pacote da rede gera `PACKET_IN` em cada switch do caminho, e o plano de controle se torna o gargalo por construção.

O objetivo deste artigo é submeter esse controlador a testes de estresse e de exaustão e quantificar seus limites de vazão, uso de CPU, perda e latência, usando o Mininet [Lantz et al. 2010] com Open vSwitch [Pfaff et al. 2015] na topologia `tree, depth=2, fanout=4` e o `cbench`, em quatro experimentos: (i) handshake com conexões simultâneas; (ii) inundação de `PACKET_IN`, incluindo o `ping -f`; (iii) vazão bruta no `cbench`; e (iv) avaliação de três refatorações concorrentes, com *selectors*, *threads* e *asyncio*. As contribuições são uma metodologia reproduzível, com dados brutos preservados; a ligação de cada sintoma medido ao trecho de código responsável; e a medição das três refatorações, incluindo os casos em que a mudança de arquitetura não melhora o desempenho. A Seção 2 descreve o ambiente e a metodologia; a Seção 3 discute os resultados; a Seção 4 detalha as refatorações; e a Seção 5 conclui o trabalho.

2. Metodologia e Setup Experimental

Ambiente. Todos os experimentos rodaram em uma única máquina (Tabela 1), com controlador, Mininet, Open vSwitch e `cbench` comunicando-se pela interface de *loopback*. Como o Linux executa sobre o WSL2, uma máquina virtual leve do Windows, os valores absolutos valem como referência deste ambiente, e as conclusões se apoiam sobretudo em comparações relativas feitas nas mesmas condições.

Tabela 1. Ambiente experimental (registrado por `experimentos/ambiente.sh`).

Componente	Versão / configuração
Processador	Intel Core i5-12500H (12ª geração), 8 núcleos e 16 threads visíveis
Memória e sistema	7,6 GiB; Ubuntu 26.04 LTS sobre WSL2, kernel Linux 6.18
Python	3.14.4, com GIL habilitado; sem <code>uvloop</code>
Emulação	Mininet 2.3.0 e Open vSwitch 3.7.1 (OpenFlow 1.0)
Medição	<code>cbench</code> , <code>tcpdump</code> 4.99.6 e <code>top</code> (<code>procs-ng</code> 4.0.4)

Topologia e controladores. A topologia `tree, depth=2, fanout=4` tem um switch raiz ligado a quatro switches de borda, cada um com quatro hosts: 5 switches

Open vSwitch e 16 hosts (h1 a h16), com OpenFlow 1.0 e controlador remoto em 127.0.0.1:6653. O controlador base é usado sem modificações; as três refatorações (Seção 4) têm exatamente o mesmo comportamento de *hub*, de modo que as diferenças medidas decorrem só da arquitetura. O Experimento 2 usa o controlador no modo padrão, que imprime uma linha por PACKET_IN, como na demonstração; os demais usam `--quiet`, para medir a arquitetura e não a escrita no terminal. O controlador base registra a cada segundo uma linha [STATS] com os PACKET_IN processados.

Instrumentação. A CPU de cada processo é amostrada a cada 0,5 s em `/proc/<pid>/stat` (tempo de usuário mais sistema), a mesma grandeza do `top`, em percentual de um núcleo; em paralelo, `top -b -d 1 -p <pid>` grava a saída do `top`. As fases de cada experimento são marcadas no tempo, e nas médias por fase descarta-se 1 s em cada borda. Antes de cada execução, os scripts abortam se já houver um processo escutando a porta 6653 e só prosseguem quando o controlador iniciado está escutando; essa verificação foi introduzida porque, em uma execução preliminar, um processo remanescente respondeu no lugar do controlador sob teste, e aquelas medições foram descartadas.

Experimento 1. No cenário do enunciado, o controlador base é iniciado e, em seguida, a árvore, registrando-se o log do controlador (5 rodadas). No comparativo, N switches ($N \in \{1, 2, 4, 5, 8, 16\}$; $N=5$ em árvore e os demais em cadeia, o que não afeta o handshake) são criados sem controlador; o controlador é iniciado, o `tcpdump` captura a porta 6653 e uma única transação `ovs-vsctl` aponta todos os switches para ele, provocando conexões simultâneas (5 rodadas por controlador e por N). Da captura extraem-se, por switch, a *latência total* (do primeiro SYN da rodada ao FEATURES_REPLY do switch, incluindo reconexões) e o *atraso do HELLO* (do SYN da conexão bem-sucedida ao primeiro byte enviado pelo controlador), que isola o tempo de espera para ser atendido; contam-se também SYNs e conexões abortadas.

Experimento 2. O roteiro do enunciado (10 s ocioso, `pingall`, h1 `ping -f h16` por 30 s) é executado com monitoramento da CPU do controlador e do `ovs-vswitchd` e do estado das conexões. Como o `ping -f` adapta sua taxa às respostas, a curva de saturação usa injeção em taxa fixa: após um `pingall`, 14 patamares de 15 s (pausas de 3 s), de 200 a 20 000 ICMP/s nominais, distribuídos entre os pares $h_i \rightarrow h(17 - i)$ com até 200 requisições/s por processo `ping -i`.

Experimentos 3 e 4. O `cbench` emula switches que enviam PACKET_IN (100 000 MACs de origem por switch) e mede as respostas; no modo *latência* (padrão) cada switch mantém uma requisição pendente, e no modo *throughput* (`-t`) envia sem esperar. Usa-se o comando do enunciado, `cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s <S>`: 10 laços de 10 s, dos quais o primeiro é aquecimento, de modo que média e desvio-padrão vêm de 9 laços. Uma única bateria alimenta os dois experimentos: CPU em repouso de cada controlador (10 s), modo *latência* com $S \in \{1, 2, 4, 5, 6, 7, 8, 16\}$ ($S=8$ é o caso do enunciado) e modo *throughput* com $S \in \{7, 8\}$.

3. Resultados Experimentais e Discussão

3.1. Experimento 1: handshake com múltiplas conexões

Inicialização da rede. Nas 5 rodadas, o `Controlador-v2.py` concluiu o handshake com os 5 switches da árvore. O terminal mostra um comportamento estritamente se-

quencial: para cada switch aparece a conexão TCP, o HELLO recebido, o FEATURES_REQUEST enviado e o FEATURES_REPLY com o DPID e o número de portas, e só então a conexão seguinte é aceita (trecho abaixo, rodada 1). Cada handshake levou entre 1,9 e 3,9 ms, com cerca de 11 ms entre um switch e o próximo, e os 5 ficaram conectados em aproximadamente 50 ms. Mensagens que chegam durante a negociação, como o PORT_STATUS do switch 5, são descartadas (“ignorada durante o handshake”). Logo em seguida começa o fluxo de PACKET_IN gerado pelo tráfego IPv6 de descoberta de vizinhos dos hosts. A primeira linha “[ERRO] Handshake falhou” de cada log não é um switch: é a sonda com que o Mininet verifica se a porta do controlador está aberta.

```
[18:17:40.902] [+] Conexão TCP estabelecida com o switch: 127.0.0.1:60952
[18:17:40.902] <<< HELLO de 127.0.0.1:60952 (xid=807)
[18:17:40.902] >>> FEATURES_REQUEST para 127.0.0.1:60952
[18:17:40.905] <<< FEATURES_REPLY: DPID=00000000000000001 ... portas=5 (3.90 ms)
[18:17:40.906] [OK] Handshake concluído com DPID 1 (1 switch(es) conectado(s))
...
[18:17:40.952] [OK] Handshake concluído com DPID 5 (5 switch(es) conectado(s))
```

Nesse cenário o atraso é pequeno porque o Mininet cria os switches um após o outro. O gargalo aparece quando as conexões chegam juntas, como mostra a Figura 1. Com o controlador base, a latência total de handshake cresce de 0,3 s com 1 switch para 1,1 s com 5 e 4,3 s com 16, e o atraso até o controlador enviar o HELLO, que mede apenas a espera para ser atendido, passa de 0,8 ms para cerca de 400 ms (N=4 a 8) e 1,06 s (N=16). Com 16 switches, observam-se em média 42,4 SYNs e 9,2 conexões abortadas pelos switches por rodada: as conexões que esperam demais são desfeitas pelo Open vSwitch e refeitas após um intervalo de espera. Nas três refatorações, nenhuma conexão foi abortada, o número de SYNs igualou o de switches e o atraso do HELLO ficou abaixo de 1,2 ms (selectors e asyncio) e de 9 ms (threads, que precisa criar uma thread por conexão). A latência total de 100 a 260 ms das threads, o valor de 0,3 s do base com um único switch e o pico isolado do asyncio em N=5 vêm do próprio Open vSwitch: as capturas mostram o controlador enviando o FEATURES_REQUEST em menos de 1 ms e o switch respondendo em degraus de cerca de 100 ou 500 ms.

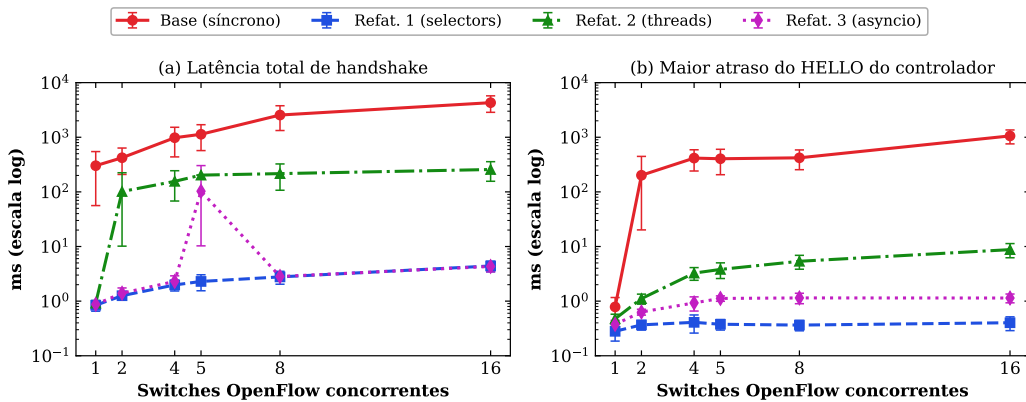


Figura 1. Experimento 1: (a) latência total de handshake e (b) maior atraso do HELLO do controlador por rodada, em função do número de switches que conectam simultaneamente (média $\pm$ desvio-padrão de 5 rodadas; N=5 usa a topologia em árvore).

Gargalo. O atraso nasce no laço principal do controlador base, que executa `accept()` e, em seguida, `handshake()` de forma síncrona: enquanto espera o HELLO e o FEATURES_REPLY de um switch, `recv_exact()` gira em espera ocupada sobre um socket não bloqueante e nenhuma outra conexão é aceita ou atendida. As conexões seguintes ficam na fila de `accept` do kernel, limitada por `listen(5)`; quando ela enche, novos SYNs deixam de ser respondidos e o cliente precisa retransmiti-los, o que explica os SYNs excedentes e os atrasos de segundos. Um switch lento ou malicioso que não responda ao FEATURES_REQUEST bloqueia o controlador inteiro.

3.2. Experimento 2: estresse de table-miss via PACKET_IN

No roteiro do enunciado (Figura 2a), o `pingall` terminou sem perda em 3,6 s. Durante os 30 s de `h1 ping -f h16`, a CPU do controlador ficou em 23% em média, com máximo de 30% nas amostras de `/proc` e de 27% no `top`, contra 7,5% em ociosidade, processando cerca de 1,5 mil PACKET_IN/s; o `ovs-vswitchd` ficou em torno de 45%. O `ping -f` enviou 3 389 pacotes, com perda de 0,03% (um pacote) e RTT médio de 8,7 ms (máximo de 30 ms). Os 5 switches permaneceram conectados, sem nenhum evento de desconexão no log do Open vSwitch. A CPU não chega a 100% porque o `ping -f` é autorregulado: só envia um novo pacote quando a resposta anterior chega ou a cada 10 ms, e a passagem de cada pacote pelo controlador em três switches impõe um RTT de 8,7 ms, o que limita a carga a cerca de 113 pacotes/s. Ou seja, a própria lentidão do controlador limita a carga oferecida, e levá-lo à exaustão exige injeção em taxa fixa.

A Figura 2b mostra a curva de saturação com o controlador no modo padrão. Cada ICMP enviado gera cerca de 10 PACKET_IN, porque o *hub* não instala regras: a requisição e a resposta passam por três switches (borda, raiz e borda) e cada cópia inundada que chega a outro switch volta ao controlador. Até 2,6 mil ICMP/s (26 mil PACKET_IN/s) não há perda relevante, mas a CPU sobe de 22% para 79%. A partir de 3,2 mil ICMP/s a CPU passa de 97% e a vazão do controlador estabiliza entre 29 e 32 mil PACKET_IN/s; o excedente se acumula nas filas, o RTT médio sobe de 9 ms para 1,4 s e a perda cresce de 20% até 100% com cerca de 6 mil ICMP/s. Mesmo assim, os 5 switches permaneceram conectados em todos os patamares: o controlador continua respondendo aos ECHO_REQUEST, pois os atende no mesmo laço, e a degradação aparece como perda e latência, não como desconexão.

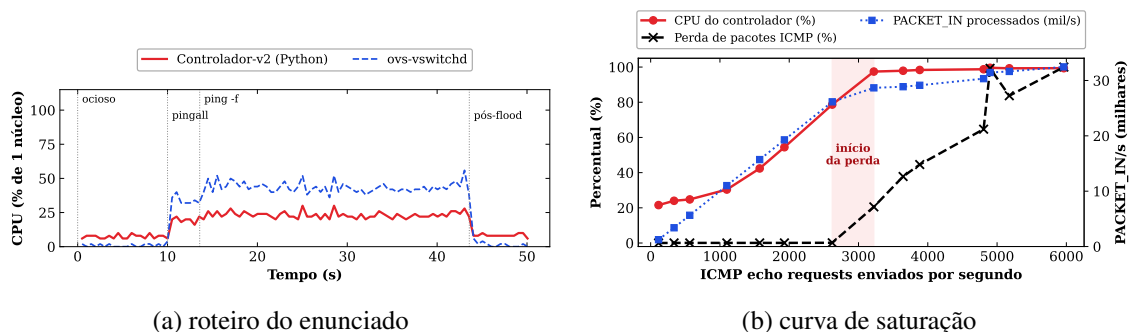


Figura 2. Experimento 2: (a) %CPU do controlador e do ovs-vswitchd durante pingall e h1 ping -f h16; (b) %CPU, perda de ICMP e PACKET_IN/s processados por taxa de ICMP enviada (patamares de 15 s).

`recv_exact()` e `struct.unpack()`. Para cada PACKET_IN, o contro-

lador base faz uma chamada `recv(8)` para o cabeçalho e outra, via `recv_exact()`, para o corpo; decodifica o cabeçalho e o corpo com `struct.unpack()`; monta o `PACKET_OUT` com três `struct.pack()` e concatenações; imprime uma linha de log; e envia com `sendall()`. A Tabela 2 mostra o custo medido de cada etapa. Ao contrário do que se poderia supor, a decodificação com `struct.unpack()` custa pouco (130 a 185 ns, pois o módulo mantém em cache as strings de formato) e responde por cerca de 1% dos $\approx 33 \mu\text{s}$ gastos por `PACKET_IN` na saturação. O que limita a vazão é a E/S: as duas syscalls de leitura e a de escrita por mensagem custam $3,4 \mu\text{s}$, contra 63 ns por mensagem quando um único `recv` traz 64 mensagens; e cada volta vazia do *polling* custa $1,8 \mu\text{s}$, porque gera uma exceção `BlockingIOError`. Como o laço varre todos os switches a cada iteração, cada mensagem útil paga também as voltas vazias dos sockets ociosos. Além disso, quando uma mensagem chega fragmentada, `recv_exact()` gira até completá-la sem atender mais nenhum switch, de modo que a vazão total fica limitada pelo switch mais lento. Por fim, o `print` por mensagem ($0,9 \mu\text{s}$ mais a escrita no terminal) é síncrono e está no caminho crítico.

Tabela 2. Custo por operação no laço de `PACKET_IN` (`microbench.py`, mínimo de 5 repetições).

Operação	Custo (ns)
<code>struct.unpack('!BBHI')</code> do cabeçalho	127
<code>struct.unpack('!IHbBx')</code> do corpo do <code>PACKET_IN</code>	185
$3 \times \text{struct.pack} + \text{concatenação}$ do <code>PACKET_OUT</code>	689
Formatação e <code>print</code> do log por <code>PACKET_IN</code>	945
<code>send + 2 \times recv</code> (enquadramento do controlador base)	3 361
<code>send + 1 \times recv</code> de 64 mensagens, por mensagem	63
<code>recv()</code> sem dados $\rightarrow$ <code>BlockingIOError</code> (uma volta do <i>polling</i>)	1 842

3.3. Experimento 3: vazão bruta com o `cbench`

Com o comando do enunciado (`-s 8`), o `cbench` não consegue estabelecer as conexões e encerra sem medir vazão, resultado que se repetiu em todas as execuções:

```
$ cbench -c 127.0.0.1 -p 6653 -m 10000 -l 10 -s 8
...
faking 8 switches offset 1 :: 10 tests each; 10000 ms per test
make_tcp_connection: connect: Operation now in progress
make_nonblock_tcp_connection :: returned -1
```

O `cbench` abre as conexões com `connect()` não bloqueante e trata como erro uma conexão que não se completa de imediato. Como o controlador base negocia um switch por vez e a fila de `accept` é limitada por `listen(5)`, a oitava conexão simultânea fica pendente e o teste aborta; com 7 switches o teste sempre completa. Durante a tentativa, a CPU do controlador chega a 100%, consumida pela espera ocupada do handshake. A maior vazão registrada foi, portanto, obtida com 7 switches: $108\,255 \pm 2\,215$ respostas/s em média (máximo de 111 192 em um laço):

```
RESULT: 7 switches 9 tests min/max/avg/stddev =
104967.88/111191.79/108255.45/2214.54 responses/s
```

A Figura 3 mostra que a vazão é estável ao longo dos laços e cresce de 30 mil (1 switch) para 108 mil respostas/s (7 switches), com a CPU indo de 33% a 87%: com mais switches, cada volta do laço encontra mais mensagens prontas e desperdiça menos voltas vazias. No modo *throughput* ($-\tau$), em que os switches não esperam resposta, a vazão cai para 47 214 respostas/s com 7 switches e CPU de 99,8%.

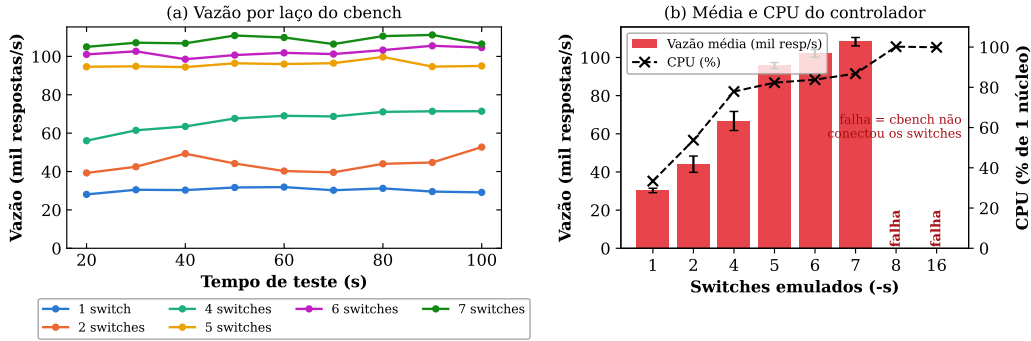


Figura 3. Experimento 3: vazão do controlador base no cbench ($-\tau$ 10000 $-\tau$ 10) por laço de teste (a) e média $\pm$ desvio-padrão com %CPU (b), por número de switches emulados.

Comparação com controladores de produção. Em medições com o cbench publicadas para o modo *throughput*, com 64 switches e uma única thread, o Beacon atingiu 1,35 milhão de respostas/s, o NOX 828 mil, o Maestro 420 mil e o Floodlight 135 mil, enquanto os controladores em Python, POX e Ryu, ficaram em 35 mil e 20 mil; com 12 núcleos, o Beacon chegou a 12,8 milhões [Erickson 2013]. Em outra comparação, ONOS respondeu a 400–500 fluxos/ms (400–500 mil/s), OpenDaylight e Beacon a até 100 mil/s, e NOX, POX e Ryu ficaram entre os piores [Zhu et al. 2019]. Hardware, versões e aplicações diferem, o que impede comparar os valores diretamente, mas a diferença de escala é clara: o controlador base sustenta 47 mil respostas/s no modo *throughput* e não aceita sequer 8 switches, enquanto os controladores de produção atendem dezenas de switches e escalam com o número de núcleos. Os fatores que justificam a diferença são: (i) Python interpretado e o GIL, que impede paralelismo real de CPU entre threads [Beazley 2010]; (ii) uma única thread, sem uso de múltiplos núcleos; (iii) handshake síncrono e `listen(5)`, que limitam o número de conexões; (iv) várias syscalls por mensagem, sem leitura nem escrita em lote, justamente a otimização que levou o Beacon a 1,35 milhão de respostas/s [Erickson 2013]; (v) *polling* com espera ocupada, cujo custo cresce com o número de switches; e (vi) log síncrono no caminho crítico. Como a aplicação é trivial (um *hub*), a diferença vem da arquitetura, e não da lógica de controle.

3.4. Experimento 4: avaliação das refatorações

A Tabela 3 e a Figura 4 comparam o controlador base às três refatorações (Seção 4) nas mesmas condições. Todas resolvem os problemas de conexão: aceitam 8 e 16 switches no cbench, negociam handshakes concorrentes em milissegundos (Seção 3.1) e não consomem CPU em repouso, contra 2,5% do base, que desperta a cada 1 ms para varrer os sockets. No modo *throughput*, as três passam de 970 mil respostas/s com 8 switches, mais de 20 vezes o que o base alcança com 7, porque cada `recv` de 64 KiB traz dezenas de mensagens, que são respondidas em um único `send`.

Tabela 3. Experimento 4: vazão no cbench (respostas/s, média de 9 laços) e %CPU.

Controlador	Modo latência ($-s$)				Modo $-t$	CPU (%)	
	1	7	8	16	8	repouso	carga ($s = 7$)
Base (síncrono)	30 292	108 255	falha	falha	falha	2,5	87
Refat. 1 (selectors)	28 965	122 984	126 458	125 386	1 068 896	0,0	97
Refat. 2 (threads)	19 647	10 088	9 819	8 764	1 096 346	0,0	153
Refat. 3 (asyncio)	11 772	19 709	20 647	21 168	970 569	0,0	98

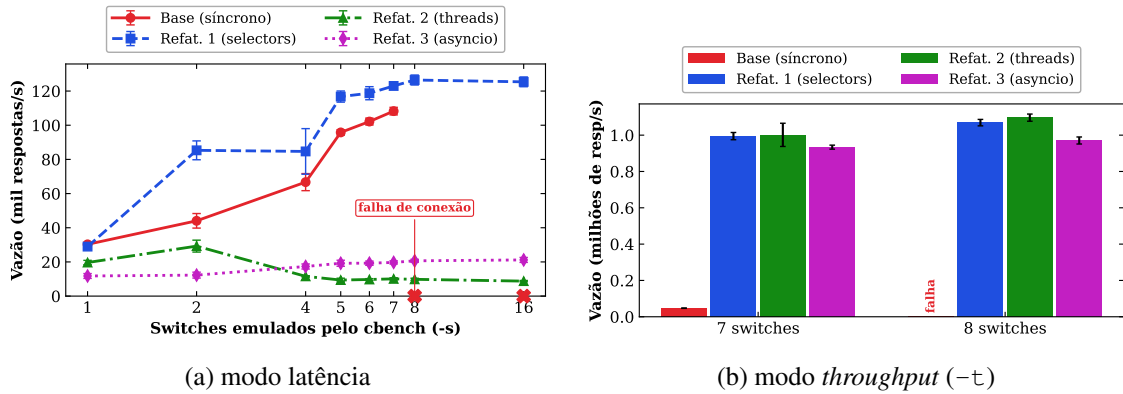


Figura 4. Experimento 4: vazão no cbench do controlador base e das três refato-rações.

No modo latência, que é o do enunciado, o resultado depende da arquitetura. Só a refatoração com *selectors* supera o base em todos os casos: 14% a mais com 7 switches e cerca de 125 mil respostas/s com 8 e 16 switches, que o base não suporta. A versão com *threads* cai para cerca de 10 mil respostas/s a partir de 4 switches, com CPU acima de 150%: como cada switch tem uma única mensagem pendente, cada resposta exige que o GIL passe de uma thread a outra, e o tempo se perde nessa disputa [Beazley 2010]. A versão com *asyncio* fica entre 12 e 21 mil respostas/s, com a CPU saturada: sem mensagens acumuladas para processar em lote, o custo fixo do *event loop* por mensagem (callbacks, *futures* e a troca de tarefa entre leitor e despachante) domina.

4. Proposta de Refatoração Arquitetural

As três propostas foram implementadas e medidas (Seção 3.4). Elas mantêm o comportamento do controlador base e compartilham um módulo `OpenFlow 1.0 (of10.py)` com duas mudanças que atacam os gargalos da Seção 3. A primeira é o enquadramento sobre buffer: em vez de `recv_exact()`, cada conexão acumula o que chega em um `bytearray` e extrai todas as mensagens completas de uma vez, de modo que um único `recv(65536)` entrega dezenas de `PACKET_IN` e uma mensagem incompleta simplesmente aguarda o próximo evento, sem espera ocupada. A segunda é o envio agregado: as respostas de um lote são concatenadas e enviadas em uma única chamada. O `PACKET_OUT` também passa a ser montado em um só `pack`.

Refatoração 1: multiplexação de E/S com *selectors*. O laço de *polling* é substituído por `selector.select()`, que usa o `epoll` do Linux e bloqueia no ker-

nel até que algum socket tenha dados, eliminando a espera ocupada e o custo das voltas vazias. O handshake deixa de ser síncrono e vira uma máquina de estados por conexão: ao aceitar, o controlador apenas enfileira seu HELLO; o FEATURES_REQUEST é enviado quando chega o HELLO do switch, e o DPID é registrado quando chega o FEATURES_REPLY, sem que um switch bloqueie outro. Cada conexão tem buffers de leitura e de escrita, e a leitura é suspensa quando há mais de 1 MiB pendente para envio (*backpressure*). O backlog passa a 128.

Refatoração 2: uma thread por switch. A thread principal executa apenas `accept()` bloqueante e entrega cada conexão a uma thread dedicada, que usa `recv()` bloqueante com o enquadramento sobre buffer. Os handshakes ocorrem em paralelo e um switch lento afeta só a sua thread; enquanto não há dados, a thread dorme no kernel e libera o GIL. É a mudança mais simples a partir do código original, mas o GIL serializa o processamento (Seção 3.4); para ganhar paralelismo de CPU, a evolução natural é o `multiprocessing`.

Refatoração 3: corrotinas com `asyncio` e fila de eventos. Cada switch é atendido por uma corrotina leitora que, a cada `await reader.read()`, devolve o controle ao *event loop*, e por um despachante; entre os dois, uma `asyncio.Queue` limitada a 64 lotes separa a E/S da lógica de controle. Essa fila permite acoplar aplicações (aprendizado de MAC, firewall) sem tocar no código de rede, aplica *backpressure* (a leitura pausa se o processamento atrasar) e, por ser por switch, impede que um switch lento atrase os demais. É o modelo mais próximo do adotado pelo Ryu, que usa *green threads* e filas de eventos entre os componentes [Ryu SDN Framework Community 2026]. Seu custo por mensagem é o mais alto dos três; o `uvloop`, *event loop* escrito em C, é o caminho natural para reduzi-lo.

5. Conclusão e Trabalhos Futuros

Os limites do `Controlador-v2.py` vêm de sua arquitetura de E/S. O handshake síncrono, aliado a `listen(5)`, faz a latência de conexão crescer até 4,3 s com 16 switches e impede o `cbench` de conectar 8 switches. O *polling* com espera ocupada e as várias `syscalls` por mensagem limitam o controlador a cerca de 30 mil `PACKET_IN/s` sob inundação, com CPU acima de 97% e perda de até 100% dos pacotes, e a 108 mil respostas/s no melhor caso do `cbench`. Já o `struct.unpack()` responde por só cerca de 1% do custo por mensagem.

Esses gargalos são exatamente os que levaram frameworks como o Ryu a adotar modelos assíncronos orientados a eventos: um laço que só desperta quando há dados, handshakes tratados como eventos independentes, leitura e escrita em lote e filas que desacoplam a rede das aplicações. As refatorações confirmam a lição, com uma nuance: a refatoração com `selectors` melhorou todos os indicadores, enquanto `threads` e `asyncio` resolveram as conexões e a ociosidade, mas perderam vazão quando cada switch tem uma única mensagem pendente. A escolha do modelo de concorrência deve, portanto, considerar o custo por mensagem e o regime de carga, e não apenas a eliminação do bloqueio.

Como trabalhos futuros: instalar regras com `FLOW_MOD` (aprendizado de MAC), reduzindo os `PACKET_IN`; usar `multiprocessing` para contornar o GIL; avaliar o `uvloop` e o Python sem GIL; e repetir as medições em máquina física.

Referências

- Beazley, D. (2010). Understanding the Python GIL. PyCon 2010, Atlanta. <https://www.dabeaz.com/python/UnderstandingGIL.pdf>.
- Berde, P., Gerola, M., Hart, J., Higuchi, Y., Kobayashi, M., Koide, T., Lantz, B., O'Connor, B., Radoslavov, P., Snow, W., and Parulkar, G. (2014). ONOS: Towards an open, distributed SDN OS. In *3rd Workshop on Hot Topics in Software Defined Networking (HotSDN)*, pages 1–6.
- Erickson, D. (2013). The Beacon OpenFlow controller. In *2nd ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking (HotSDN)*, pages 13–18.
- Kreutz, D., Ramos, F. M. V., Veríssimo, P. E., Rothenberg, C. E., Azodolmolky, S., and Uhlig, S. (2015). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1):14–76.
- Lantz, B., Heller, B., and McKeown, N. (2010). A network in a laptop: Rapid prototyping for software-defined networks. In *9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*.
- McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., and Turner, J. (2008). OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2):69–74.
- Medved, J., Varga, R., Tkacik, A., and Gray, K. (2014). OpenDaylight: Towards a model-driven SDN controller architecture. In *IEEE 15th International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*.
- Open Networking Foundation (2009). OpenFlow switch specification, version 1.0.0 (wire protocol 0x01). <https://opennetworking.org/wp-content/uploads/2013/04/openflow-spec-v1.0.0.pdf>.
- Pfaff, B., Pettit, J., Koponen, T., Jackson, E., Zhou, A., Rajahalme, J., Gross, J., Wang, A., Stringer, J., Shelar, P., Amidon, K., and Casado, M. (2015). The design and implementation of Open vSwitch. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 117–130.
- Ryu SDN Framework Community (2026). Ryu: Component-based software defined networking framework. <https://ryu-sdn.org>. Acesso em: set. 2026.
- Tootoonchian, A., Gorbunov, S., Ganjali, Y., Casado, M., and Sherwood, R. (2012). On controller performance in software-defined networks. In *2nd USENIX Workshop on Hot Topics in Management of Internet, Cloud, and Enterprise Networks and Services (Hot-ICE)*.
- Zhu, L., Karim, M. M., Sharif, K., Li, F., Du, X., and Guizani, M. (2019). SDN controllers: Benchmarking & performance evaluation. arXiv:1902.04491 [cs.NI].